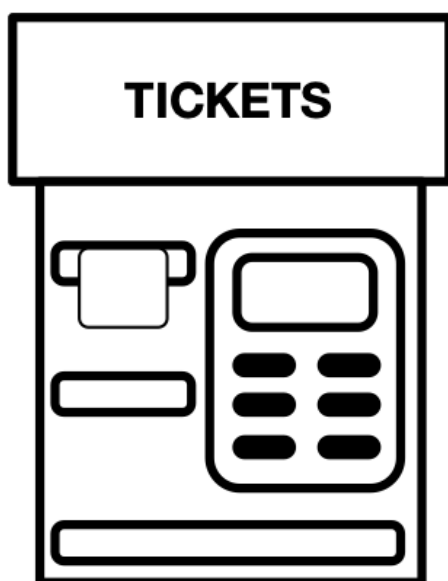




MÁQUINA DE VENDA DE BILHETES
(*Ticket Machine*)

Projeto
de
Laboratório de Informática e Computadores
2025 / 2026 verão

Trabalho Realizado por:
54035: Beatriz Leme 53799: Leonardo Silva 53652: Miguel Ruivo

Docentes: David Velez, Nuno Sebastião, Pedro Miguens, Rogério Rebelo, Rui

Duarte

12 de junho de 2026

Conteúdo

1	Descrição	1
2	Arquitetura do Sistema	3
2.1	<i>Keyboard Reader</i>	3
2.1.1	KeyDecode	4
2.1.2	RingBuffer	7
2.1.3	RingBufferControl	8
2.1.4	KeyTransmitter	10
2.2	<i>Coin Acceptor</i>	11
2.3	<i>Port Expander LCD</i>	12
2.4	<i>Port Expander Ticket Dispenser</i>	13
2.4.1	SerialReciver	14
2.5	<i>Control</i>	14
2.5.1	HAL	14
2.5.2	KBD	15
2.5.3	TUI	15
2.5.4	LCD	15
2.5.5	TD	15
2.5.6	SerialReciver	16
2.5.7	SerialEmitter	16
2.5.8	CoinAcceptor	16
2.5.9	CoinDeposit	16
2.5.10	FileAccess	16
2.5.11	M	16
2.5.12	Stations	16
2.5.13	TicketMachineAPP	17
3	Resultados Experimentais	18
3.1	Simulação Funcional do Hardware	18
3.2	Simulação do Software	19
4	Conclusões	21
5	Trabalho Futuro	21
	Referências	23
A	Ticket Machine - Hardware	24
A.1	TicketMachine.vhd	24
A.2	Atribuição de Pinos	26

B	Ticket Machine - Software (<i>Control</i>)	27
B.1	Ticket Machine - App	27
B.2	M	35
B.3	TUI	36
B.4	Coin Acceptor	40
B.5	Coin Deposit	41
B.6	Statistics	42
B.7	File Access	42

Lista de Figuras

1	Diagrama de blocos da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	1
2	Arquitetura do sistema que implementa a máquina de venda de bilhetes (<i>Ticket Machine</i>)	3
3	Diagrama de blocos do <i>Keyboard Reader</i>	4
4	Diagrama de blocos	4
5	Bloco <i>Key Decode</i> - Diagrama temporal	5
6	Diagrama temporal	6
7	Diagrama de blocos	7
8	Diagrama de blocos	8
9	Diagrama de blocos	10
10	Composição do Datapath	11
11	Bloco <i>Coin Acceptor</i>	11
a	Diagrama de blocos	11
b	Diagrama temporal	11
12	Diagrama de blocos do módulo <i>Port Expander LCD (PELCD)</i>	12
13	Protocolo de comunicação com o módulo Port Expander LCD (PELCD)	12
14	Diagrama de blocos do módulo <i>Port Expander Ticket Dispenser (PETD)</i>	13
15	Protocolo de comunicação com o Port Expander Ticket Dispenser (PETD)	13
16	Protocolo de comunicação com o mecanismo de dispensa de bilhetes (Ticket Dispenser)	14
17	Diagrama de blocos do bloco Serial Receiver	14
18	Diagrama lógico da Máquina de Venda de Bilhetes (<i>Ticket Machine</i>)	15
19	Simulação do preenchimento da memória RAM do <i>Ring Buffer</i> e ativação dos sinais <i>empty</i> e <i>full</i>	18
20	Ativação e apresentação do menu do modo de Manutenção no simulador de hardware.	20

Lista de Tabelas

1	Codificação do valor facial das moedas	12
2	Atribuição de pinos para a função F [4]	26

Lista de Algoritmos

1	TicketMachineAPP	27
2	M	35
3	TUI	36
4	Coin Acceptor	40
5	Coin Deposit	41
6	File Access	42

1 Descrição

Este projeto implementa uma máquina de venda de bilhetes (*Ticket Machine*), que permite a aquisição de bilhetes de comboio. O percurso é definido pela estação de origem, local de compra do bilhete e pela seleção do destino digitando o identificador da estação ou através das teclas ↑ e ↓, sendo exibido no ecrã além do identificador da estação de destino o preço e o tipo de bilhete (ida ou ida/volta). A ordem de aquisição é dada através da pressão da tecla de confirmação, sendo impressa uma unidade do bilhete exibido no ecrã. A máquina não realiza trocos e só aceita moedas de: 0,05€; 0,10€; 0,20€; 0,50€; 1,00€; e 2,00€. Para além do modo de Dispensa, o sistema tem mais um modo de funcionamento designado por Manutenção, que é ativado por uma chave de manutenção. Este modo permite o teste da máquina de venda de bilhetes, bem como permite iniciar e consultar os contadores de bilhetes e moedas. A máquina de venda de bilhetes é constituída pelo sistema de gestão (designado por *Control* na Figura 1) e pelos seguintes periféricos: um teclado de 16 teclas, um moedeiro (designado por *Coin Acceptor*), um ecrã *Liquid Cristal Display* (LCD) de duas linhas de 16 caracteres, um mecanismo de dispensa de bilhetes (designado por *Ticket Dispenser*) e uma chave de manutenção (designada por *M*) que define se a máquina de venda de bilhetes está em modo de Manutenção, conforme o diagrama de blocos apresentado na Figura 1.

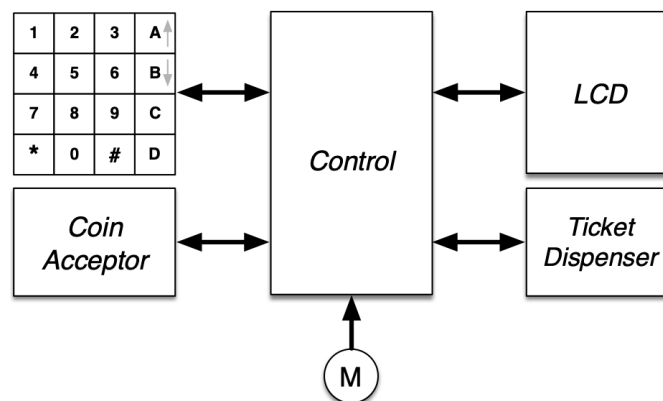


Figura 1: Diagrama de blocos da Máquina de Venda de Bilhetes (*Ticket Machine*)

Sobre o sistema podem-se realizar as seguintes ações em modo Venda:

- **Consulta e venda** A consulta de um bilhete é realizada digitando o identificador da estação de destino ou listando-a através das teclas ↑ e ↓. O processo de compra do bilhete inicia-se premindo a tecla '#'. Durante a inserção do respetivo valor monetário, é possível alterar o tipo de bilhete (ida ou ida/volta) premindo a tecla '*', com a consequente alteração do preço da viagem afixado no *LCD*, duplicando o valor no caso de ida/volta. Durante a compra ficam afixados no *LCD* as informações referentes ao bilhete pretendido até que o mecanismo de impressão de bilhetes confirme que a impressão já foi realizada e a recolha do bilhete efetuada. A compra pode ser cancelada premindo a tecla '#', devolvendo as moedas inseridas.

Sobre o sistema podem-se realizar as seguintes ações em modo Manutenção:

- **Teste** – Esta opção do menu permite realizar um procedimento de consulta e venda de um bilhete, sem introdução de moedas e sem esta operação ser contabilizada como uma aquisição.
- **Consulta Bilhetes** – Premindo a tecla 'A', em seguida através das teclas ↑ e ↓, visualizam os contadores do número de bilhetes vendidos.
- **Consulta Moedas** – Premindo a tecla 'B', em seguida através das teclas ↑ e ↓, visualizam os contadores das moedas no cofre.
- **Iniciar os contadores de moedas e bilhetes** – Premindo a tecla 'C' e em seguida a tecla '*', o sistema de gestão coloca os contadores de moedas e bilhetes a zero, iniciando um novo ciclo de contagem.
- **Desligar** – Premindo a tecla 'D' permite desligar o sistema, que encerra apenas após a confirmação do utilizador, ou seja, o programa termina e as estruturas de dados, contendo a informação dos contadores de moedas e da lista de bilhetes vendidos, são armazenadas de forma persistente em dois ficheiros de texto. A informação em cada ficheiros deve estar organizada por linha, em que os campos de dados são separados por “;”, com o respetivo formato: 'COIN_VALUE;NUMBER_COINS' (valor facial; número de moedas) e 'PRICE;SOLD_TICKETS;STATION_NAME' (preço; número de bilhetes vendidos para a estação; nome da estação). Os dois ficheiros devem ser carregados para o sistema no seu processo de arranque.

Nota: A inserção de informação através do teclado tem o seguinte critério: *i*) se não for premida nenhuma tecla num intervalo de cinco segundos o comando em curso é abortado; *ii*) quando o dado a introduzir é composto por mais que um dígito, são considerados apenas os últimos dígitos, a inserção realiza-se do dígito de maior peso para o de menor peso.

2 Arquitetura do Sistema

O sistema implementa uma solução híbrida de hardware e software, como apresentado no diagrama de blocos da Figura 2. A arquitetura proposta é constituída por cinco módulos principais: *i*) um leitor de teclado, designado por *Keyboard Reader*; *ii*) um módulo de interface com o *LCD*, designado por *Port Expander LCD (PELCD)*; *iii*) um módulo de interface com o mecanismo de dispensa de bilhetes (*Ticket Dispenser*), designado por *Port Expander Ticket Dispenser (PETD)*; *iv*) um moedeiro, designado por *Coin Acceptor*; e *v*) um módulo de controlo, designado por *Control*.

Os módulos *i*), *ii*) e *iii*) deverão ser implementados em hardware, o moedeiro (*iv*) será emulado utilizando quatro interruptores e três *LED*, enquanto o módulo de controlo (*v*) deverá ser implementado em software, descrito em linguagem *Kotlin* e executado num PC.

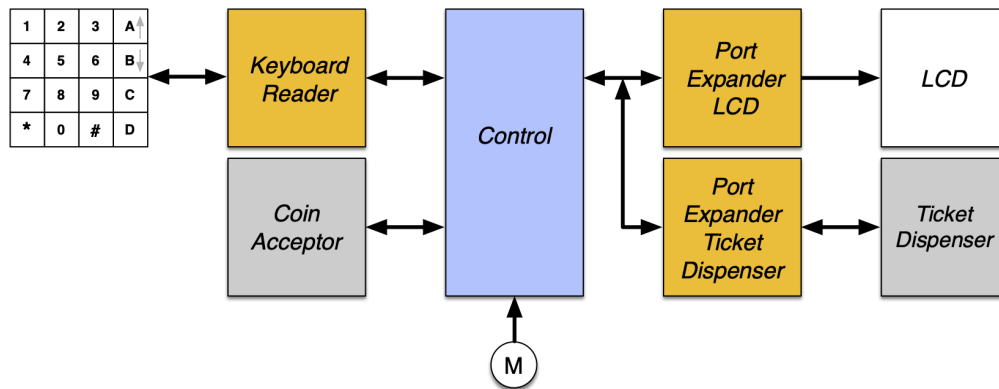


Figura 2: Arquitetura do sistema que implementa a máquina de venda de bilhetes (*Ticket Machine*)

O módulo *Keyboard Reader* é responsável pela descodificação do teclado matricial de 16 teclas, determinando qual a tecla pressionada e disponibilizando o seu código, com quatro bits, ao módulo *Control*. Caso este não esteja disponível para o receber imediatamente, o código da tecla é armazenado internamente, até ao limite de dezoito (18) códigos. O módulo *Control* processa os dados e envia a informação a apresentar no *LCD* através do módulo *PELCD*. O *Ticket Dispenser* é atuado pelo módulo *Control*, através do módulo *PETD*. Por forma a minimizar o número de interligações, a comunicação entre o módulo *Control* e os módulos *PELCD* e *PETD* é realizada através de um protocolo série.

2.1 Keyboard Reader

O módulo *Keyboard Reader* [1] é constituído por três blocos principais: *i*) o descodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*). Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

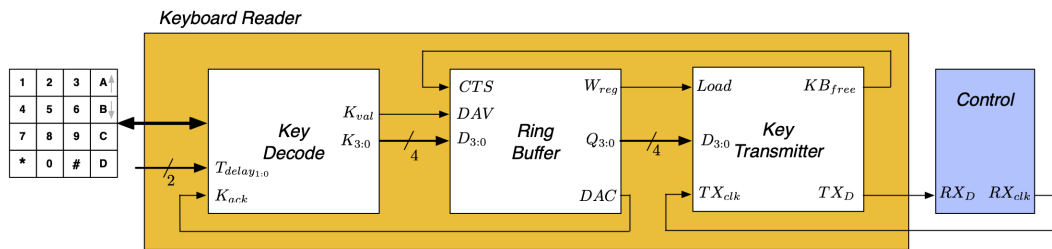


Figura 3: Diagrama de blocos do *Keyboard Reader*

2.1.1 KeyDecode

O bloco Key Decode tem como principal função implementar um decodificador em hardware para um teclado matricial de 4×4 . Estruturalmente, este sistema divide-se em três sub-blocos essenciais:

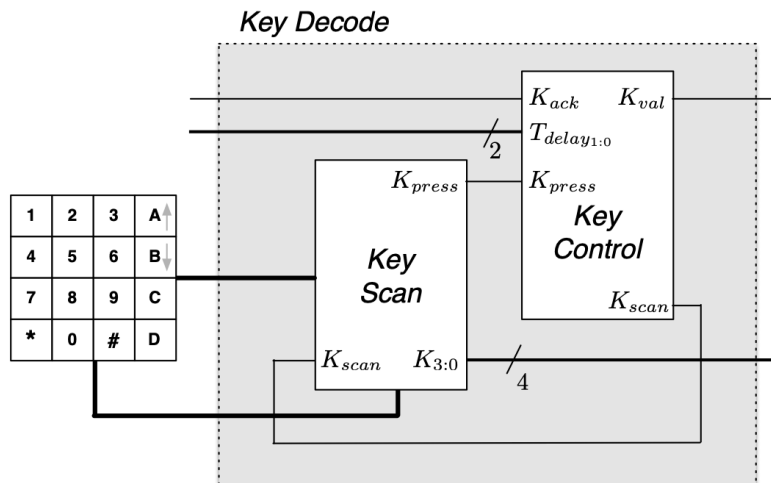


Figura 4: Diagrama de blocos

Teclado matricial: A interface física de 4×4 .

Key Scan: O módulo responsável por efetuar o varrimento contínuo das teclas, percorrendo as colunas à procura de uma linha que tenha sido ativada ao premir o botão do teclado.

Key Control: A unidade lógica que realiza o controlo do varrimento e gere o fluxo de dados do sistema que controla o Tdelay e segue o ASM presente na Figura 5.

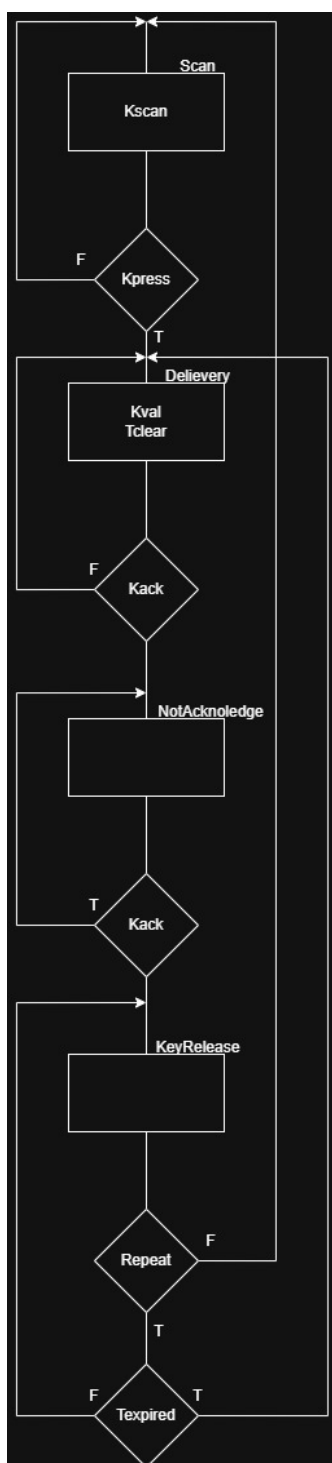


Figura 5: Bloco *Key Decode* - Diagrama temporal

Sempre que o sistema deteta a pressão de uma tecla, o sinal de saída K_{val} é ativado. Simultaneamente, o código representativo dessa tecla é disponibilizado no barramento de dados $K_{3:0}$.

Numa fase inicial, um novo ciclo de varrimento apenas se inicia após a tecla ser libertada e mediante a receção do sinal de reconhecimento K_{ack} .

Numa segunda fase do projeto, o sistema incorpora um mecanismo de repetição de tecla (auto repeat). Caso a tecla se mantenha pressionada, o sistema gera automaticamente um novo código após um período de tempo pré-determinado, definido pela variável de entrada $T_{delay_{1:0}}$. Os períodos de atraso configuráveis são os seguintes:

- 00: 500 ms
- 01: 1000 ms
- 10: 1500 ms
- 11: 2000 ms

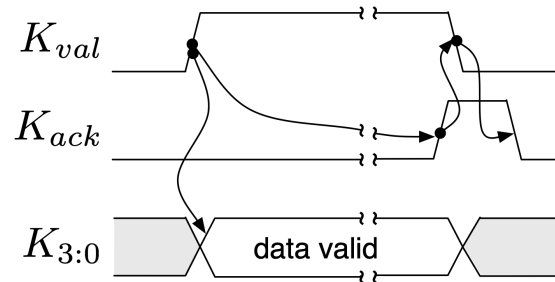


Figura 6: Diagrama temporal

Para a realização do varrimento do teclado, foi adotada a Versão I da arquitetura do bloco Key Scan. Nesta configuração, o circuito é conduzido por um oscilador que alimenta o sinal de clock de um contador síncrono. As saídas deste contador são divididas para controlar dois componentes em paralelo:

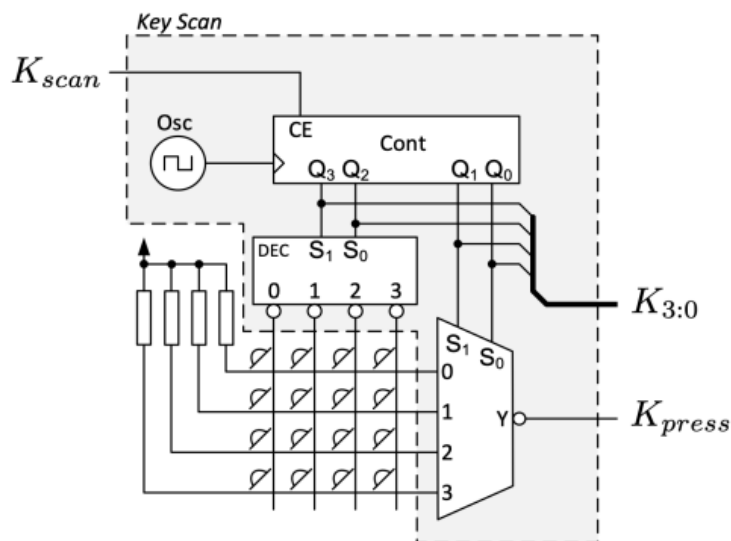


Figura 7: Diagrama de blocos

Descodificador (Decoder): Recebe parte dos bits do contador para ativar sequencialmente as linhas do teclado matricial.

Multiplexer (MUX): Utiliza os restantes bits do contador como sinais de seleção para ler o estado das colunas do teclado, canalizando o resultado para a saída K_{press} .

Esta abordagem garante um varrimento cíclico e sincronizado, permitindo que o bloco Key Control identifique de forma precisa as coordenadas da tecla acionada.

2.1.2 RingBuffer

O bloco Ring Buffer implementa uma estrutura de dados para armazenamento de teclas com disciplina FIFO, com capacidade de armazenar até dezasseis (16) palavras de quatro (4) bits. É composto por 3 sub-blocos: O Memory Address Control (MAC) que tem como objetivo indicar o endereço respetivo à operação realizada de acordo com a entrada de seleção PnG do bloco, o RingBufferControl que é a máquina de estados que vai controlar o RingBuffer tendo em conta os sinais que vai receber provenientes do KeyDecode (DAV) e do KeyTransmitter(CTS) e por fim a RAM que vai armazenar as 16 palavras de 4 bits.

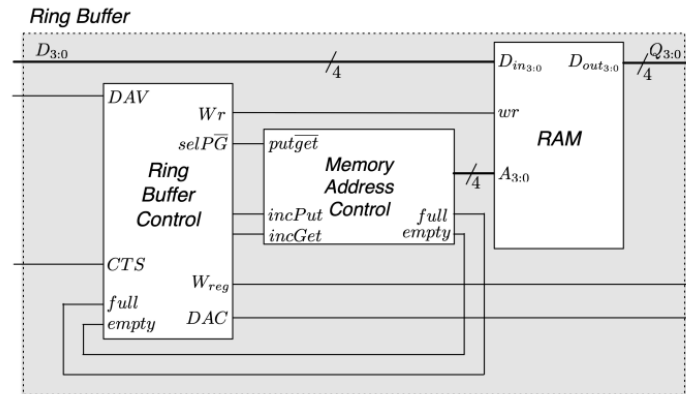


Figura 8: Diagrama de blocos

2.1.3 RingBufferControl

A coordenação do fluxo de dados e da interação com os módulos adjacentes é realizada pelo bloco Ring Buffer Control, cujo comportamento é ditado pela máquina de estados implementada. A partir do seu estado inicial (*Idle*), o sistema ramifica a sua execução consoante se trate de um processo de escrita ou de leitura:

1. Processo de Escrita (Interação com o Sistema Produtor):

- A escrita é desencadeada quando o sistema produtor (neste caso, o *Key Decode*) ativa o sinal DAV (*Data Available*).
- Se a memória não estiver cheia (condição *Full* falsa), a máquina transita para o estado *prev_Write*, ativando o sinal PnG para selecionar o endereço *putIndex* no MAC.
- No estado seguinte, *Writing*, o sistema ativa o sinal de escrita *Wr* mantendo o PnG ativo, o que efetiva o registo dos dados $D_{3:0}$ na RAM.
- Imediatamente a seguir, no estado *pos_write*, é emitido o comando *incput* (em conjunto com o PnG) para atualizar o endereço de escrita.
- Concluída esta etapa, a máquina entra no estado *ack_Producer*, ativando o sinal DAC (*Data Accepted*). Esta ação serve para informar o sistema produtor de que os dados foram guardados com sucesso.
- O sistema produtor deve então desativar o DAV; só após essa desativação é que a máquina de estados abandona o estado *ack_Producer*, regressando a *Idle* e desativando consequentemente o DAC.

2. Processo de Leitura (Interação com a Entidade Consumidora):

- A leitura de dados ocorre sempre que a entidade consumidora indica disponibilidade para receber informação, através da ativação do sinal CTS (*Clear To Send*).
- A partir do estado *Idle*, caso não haja solicitações pendentes de escrita (ou se houver uma solicitação de escrita mas a RAM estiver cheia), a máquina verifica o estado do sinal CTS.

- Se o CTS estiver ativo e houver dados disponíveis na memória (condição *Empty* falsa), a máquina avança para o estado *Reading*. Neste estado, é gerado o sinal *Wreg*.
- O sistema aguarda no estado *Reading* até que o sinal CTS seja desativado pela entidade consumidora.
- Após a transição descendente de CTS, a máquina passa para o estado *pos_Read*, onde emite o sinal *incget* para incrementar o endereço de leitura no MAC. Finalmente, retorna ao estado *Idle*, concluindo o ciclo.

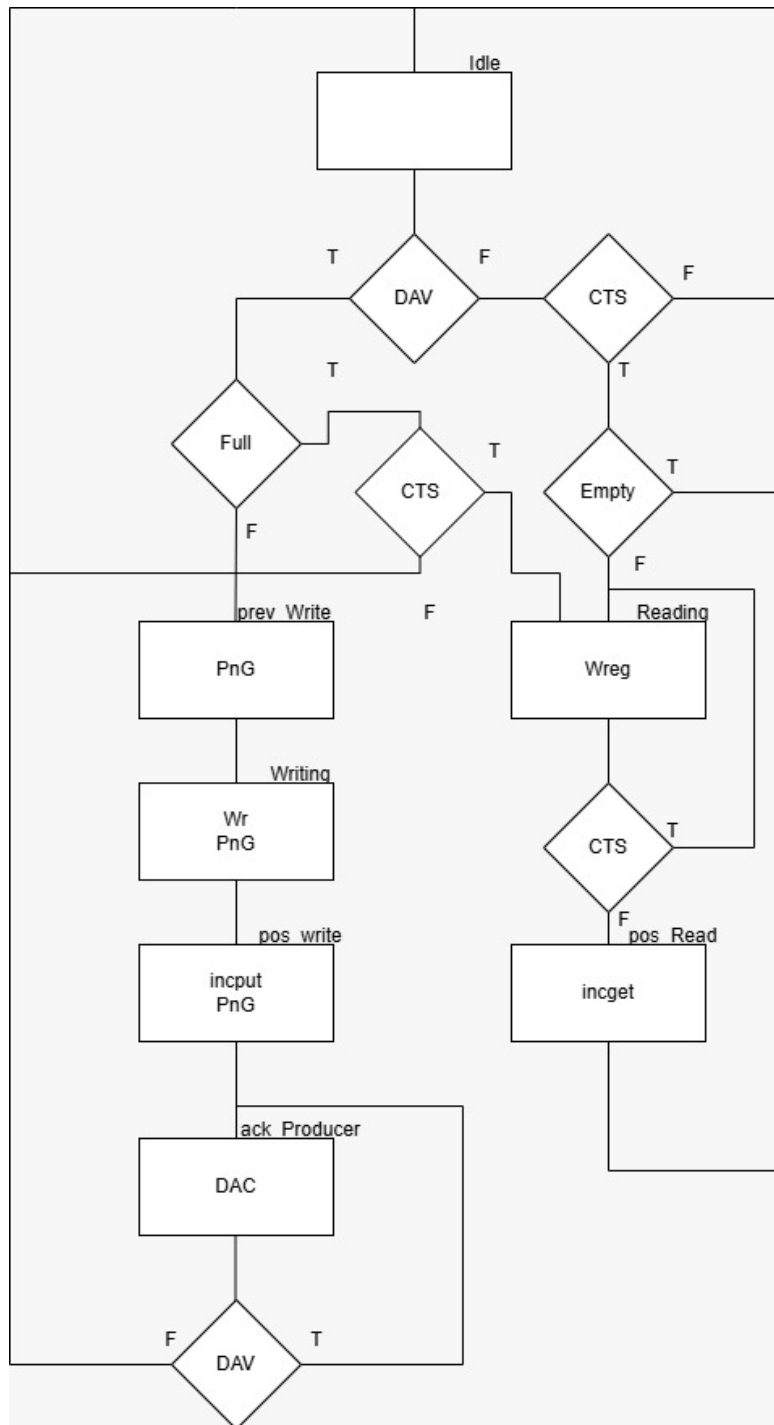


Figura 9: Diagrama de blocos

2.1.4 KeyTransmitter

O bloco Key Transmitter é responsável por atuar como uma interface de comunicação série entre o módulo produtor (responsável por registar fisicamente a tecla pressionada) e o módulo consumidor (o módulo Control central do sistema). O mesmo é composto por 2 sub-blocos

muito importantes: o Datapath que contem os componentes físicos necessários para armazenar e encaminhar os bits corretos para a saída série(a sua composição está presente na Figura 10) e o Controlo que é o que vai controlar o Datapath seguindo o ASM presente na Figura 10.

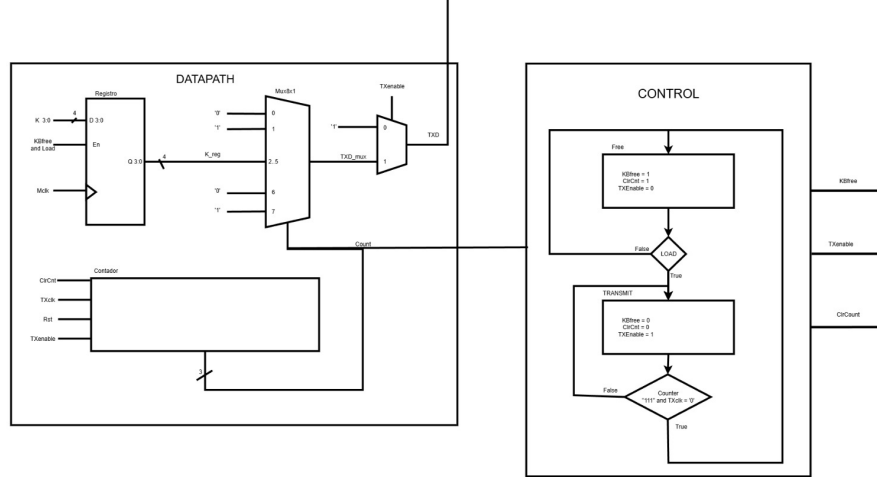


Figura 10: Composição do Datapath

2.2 Coin Acceptor

O módulo *Coin Acceptor* implementa a interface com o moedeiro, sinalizando ao módulo *Control* que o moedeiro recebeu uma moeda através da ativação do sinal *Coin*, o valor monetário da moeda inserida é codificado em $Coin_{id_{2:0}}$, conforme a codificação apresentada na Tabela 1. A entidade consumidora informa o *Coin Acceptor* que já contabilizou a moeda ativando o sinal *accept*, conforme apresentado no diagrama temporal da Figura 11b. O *Control* pode devolver as moedas inseridas no moedeiro através da ativação do sinal *eject* durante dois segundos, ou recolher as moedas presentes no moedeiro através da ativação do sinal *collect* durante dois segundos.

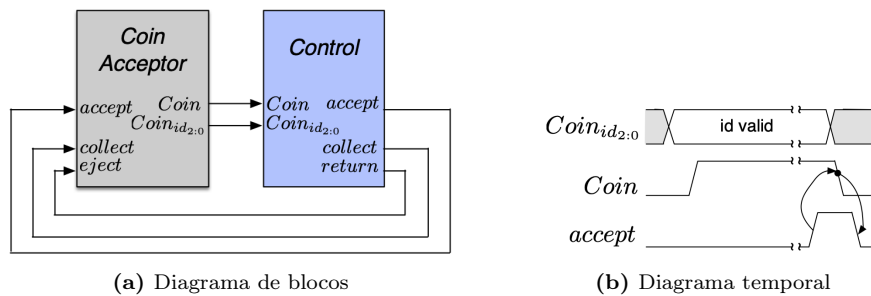


Figura 11: Bloco *Coin Acceptor*

Tabela 1: Codificação do valor facial das moedas

Moeda [€]	Codificação [$Cid_2 Cid_1 Cid_0$]
0,05	000
0,10	001
0,20	010
0,50	011
1,00	100
2,00	101

2.3 Port Expander LCD

O módulo *Port Expander LCD* [2] (*PELCD*) implementa a interface com o *LCD*, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao *LCD*, conforme representado na Figura 12.

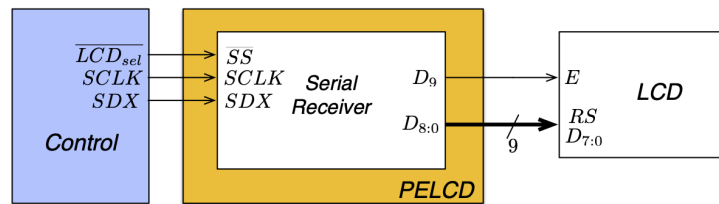


Figura 12: Diagrama de blocos do módulo *Port Expander LCD* (*PELCD*)

O módulo *PELCD* recebe em série uma mensagem constituída por dez (10) bits de informação. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura 13, em que o bit RS é o primeiro bit de informação e indica se a mensagem é de controlo ou dados. Os seguintes oito (8) bits contêm os dados a entregar ao *LCD*. O último bit contém o sinal de E responsável por registar a informação no *LCD*.

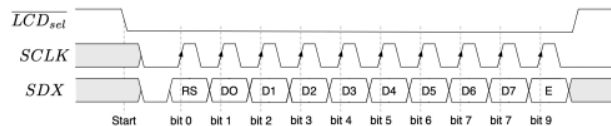


Figura 13: Protocolo de comunicação com o módulo *Port Expander LCD* (*PELCD*)

O emissor, realizado em software, quando pretende enviar uma trama para o módulo *PELCD* promove uma condição de início de trama (Start), que corresponde a uma transição descendente na linha ($LDCsel$). Após a condição de início, o módulo *PELCD* armazena os bits de dados da trama nas transições ascendentes do sinal *SCLK*. O bloco *Serial Receiver* do módulo *PELCD* é constituído por dois blocos principais: i) um registo de deslocamento; e ii) um registo de manutenção; designados por *Shift Register* e *Hold Register* respetivamente. O bloco *Serial Receiver*

deverá ser implementado com base no diagrama de blocos apresentado na Figura 17.

2.4 Port Expander Ticket Dispenser

O módulo *Port Expander Ticket Dispenser* [3] (*PETD*) implementa a interface com o mecanismo de impressão de bilhetes (*Ticket Dispenser*), realizando a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao mecanismo, conforme representado na Figura 14.

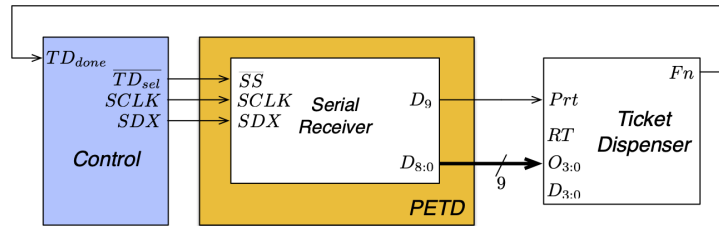


Figura 14: Diagrama de blocos do módulo *Port Expander Ticket Dispenser* (*PETD*)

O módulo PETD recebe em série uma mensagem composta por dez (10) bits de informação, segundo o protocolo de comunicação ilustrado na Figura 15. O primeiro bit de informação indica se o bilhete a imprimir é de ida ou ida/volta. Os quatro bits seguintes indicam a estação de origem, seguido da estação de destino por mais 4 bits de informação. O último bit de informação indica o início da impressão e disponibilização do bilhete. Só pode ser iniciada nova impressão quando o mecanismo de dispensa de bilhetes informar o Control que o processo terminou através do sinal Fn.

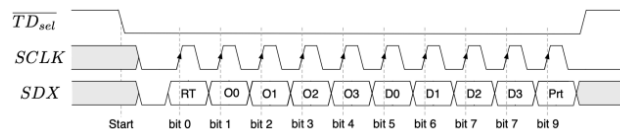


Figura 15: Protocolo de comunicação com o Port Expander Ticket Dispenser (*PETD*)

O Ticket Dispenser recebe em 4 bits o código da estação de destino (D3:0), noutros 4 bits o código da estação de origem (O3:0) a imprimir no bilhete e ainda o bit RT que define o tipo de bilhete (ida ou ida/volta). O comando de impressão do bilhete com os códigos presentes em D, O e RT é realizado pela ativação do sinal de impressão (Prt). Em resposta, o Ticket Dispenser ativa o sinal de término de execução (Fn) quando concluída a dispensa do bilhete. Os sinais Fn e Prt têm o comportamento descrito no diagrama temporal apresentado na Figura 16.

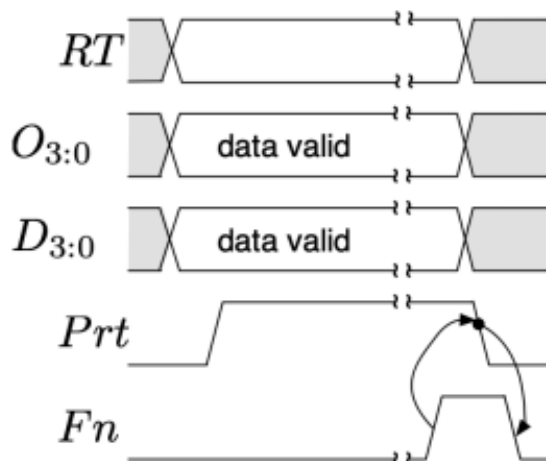


Figura 16: Protocolo de comunicação com o mecanismo de dispensa de bilhetes (Ticket Dispenser)

2.4.1 SerialReciver

Tanto o Port Expander LCD como Port Expander Ticket Dispenser utilizam o componente SerialReciver que é responsável por interpretar as tramas enviadas pelo sistema de controlo através de uma linha série.

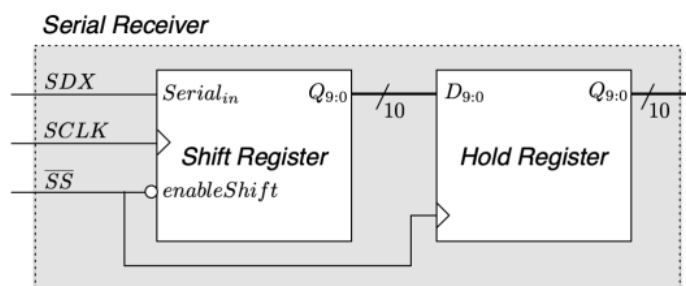


Figura 17: Diagrama de blocos do bloco Serial Receiver

2.5 Control

A implementação do módulo *Control* foi realizada em software, usando a linguagem *Kotlin* e seguindo a arquitetura lógica apresentada na Figura 18. O Controlo contém:

2.5.1 HAL

O HLA é o responsável por isolar a lógica aplicacional dos detalhes físicos do sistema, fornecendo uma interface simplificada para a manipulação dos portos de entrada e saída através do módulo *UsbPort*.

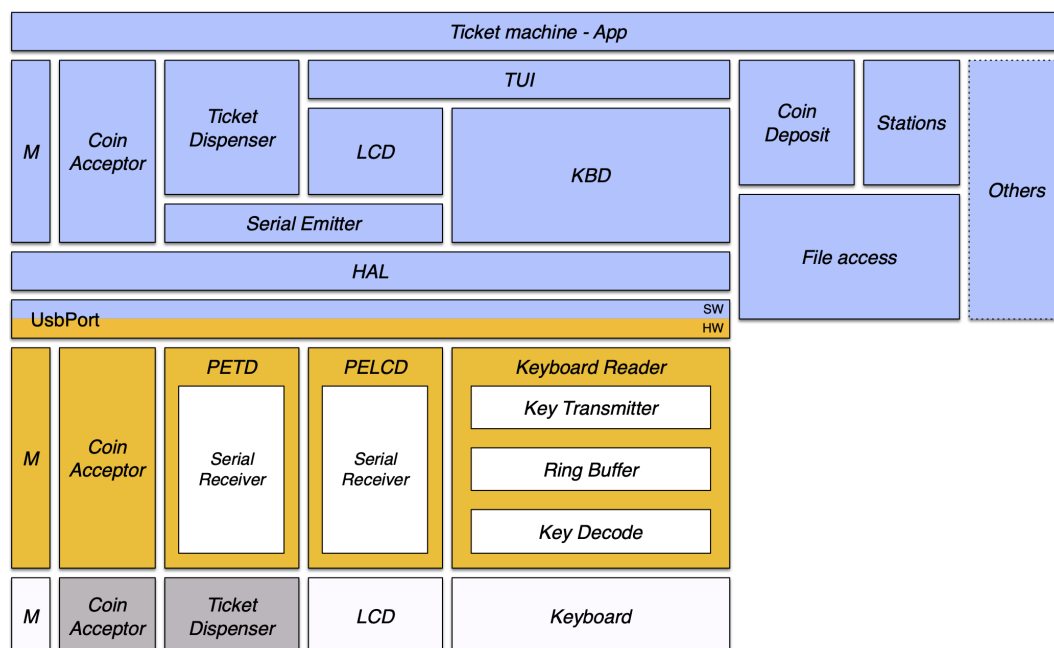


Figura 18: Diagrama lógico da Máquina de Venda de Bilhetes (*Ticket Machine*)

2.5.2 KBD

O módulo KBD a gere e leitura do teclado matricial, abstendo-se dos detalhes de comunicação física ao interagir com as camadas inferiores através dos módulos HAL e SerialReceiver. Na sua inicialização, o módulo assegura a correta configuração dos periféricos e do recetor de dados em série, preparando o sistema para a captura de eventos de pressão de teclas e está também encarregue da tradução dos sinais lógicos para os caracteres defeniudos no mesmo.

2.5.3 TUI

O módulo TUI é o responsável por gerir tudo o que o utilizador vê no ecrã e as teclas que prime, servindo de ponte entre o ecrã (LCD) e o teclado (KBD). Na inicialização, este módulo prepara o ecrã e o teclado, controlando os menus da aplicação.

2.5.4 LCD

O módulo LCD é o responsável por controlar diretamente o ecrã de 2 linhas por 16 colunas. Ele serve para traduzir o texto e os comandos que o programa envia em sinais elétricos que o ecrã consegue processar, comunicando através do módulo SerialEmitter.

2.5.5 TD

O módulo TD (Ticket Dispenser) é o componente encarregue de controlar o hardware responsável pela impressão e emissão física dos bilhetes. Ele traduz as escolhas feitas pelo utilizador no programa em dados binários estruturados que são enviados para a impressora periférica através do módulo SerialEmitter.

2.5.6 SerialReciver

O módulo SerialReceiver é o responsável por receber dados enviados em série pelo hardware (neste caso, os códigos das teclas do teclado) e transformá-los em valores numéricos que o programa consegue usar. Ele funciona através do controlo manual de um sinal de relógio (clock) e da leitura de uma linha de dados (RXD).

2.5.7 SerialEmitter

O módulo SerialEmitter traduz comandos e dados do programa para um formato em série de forma a enviá-los para os periféricos. Partilha as mesmas linhas de dados (SDX) e de relógio (SCLK) que o SerialReciver, mas consegue escolher qual o componente que vai receber a informação através de linhas de seleção dedicadas.

2.5.8 CoinAcceptor

O módulo CoinAcceptor é o responsável por controlar o hardware do moedeiro da máquina. Ele interage diretamente com os pinos da HAL para detetar quando o utilizador insere moedas, identificar o seu valor em cêntimos e comandar os switches que aceitam, devolvem ou guardam o dinheiro.

2.5.9 CoinDeposit

O ficheiro CoinDeposit define uma estrutura de dados simples (CoinEntry), que serve para gerir o inventário e a contabilidade do dinheiro dentro do sistema. Ao contrário dos módulos anteriores, este componente não mexe diretamente no hardware, servindo apenas para organizar a informação em memória.

2.5.10 FileAccess

O módulo FileAccess é o modulo encarregue de garantir a persistência de dados do sistema, permitindo guardar e carregar informações diretamente em ficheiros de texto do computador. Desta forma, a máquina não perde o histórico de bilhetes vendidos nem o stock de moedas quando é desligada.

2.5.11 M

O módulo M é um componente muito simples que serve exclusivamente para detetar o estado do interruptor físico de manutenção da máquina. Este interruptor é o que define se a máquina está em modo de funcionamento normal ou em modo de manutenção.

2.5.12 Stations

O ficheiro Stations define uma estrutura de dados simples chamada Station, que serve para guardar na memória do programa todas as informações individuais sobre cada paragem disponível na máquina. Tal como o depósito de moedas.

2.5.13 TicketMachineAPP

O módulo TicketMachineApp controla o funcionamento global da máquina de bilhetes, ligando todos os módulos criados anteriormente (moedeiro, ecrã, teclado, ficheiros e impressora) e gerindo os passos que o utilizador tem de dar para comprar um bilhete.

3 Resultados Experimentais

3.1 Simulação Funcional do Hardware

Para verificar o correto funcionamento dos módulos implementados em VHDL, recorreu-se ao ModelSim para executar as respetivas simulações funcionais. Foram desenvolvidas *testbenches* para os diferentes componentes do sistema, permitindo estimular os sinais de entrada e observar os correspondentes sinais de saída e de controlo.

Durante as simulações foram analisados, entre outros aspetos, o funcionamento das máquinas de estados, os protocolos de comunicação entre os diferentes módulos, o armazenamento e a transmissão dos dados, a ativação dos sinais de validação e confirmação e a correta receção das tramas série.

Foram realizadas simulações individuais dos módulos *Key Decode*, *Ring Buffer*, *Key Transmitter*, *Coin Acceptor*, *Port Expander LCD* e *Port Expander Ticket Dispenser*, bem como testes de integração entre os blocos que constituem o sistema.

Como exemplo dos testes realizados, apresenta-se na Figura 19 a simulação do armazenamento de dados na memória RAM utilizada pelo *Ring Buffer*. Este módulo implementa uma estrutura de armazenamento com disciplina FIFO (*First In, First Out*), com capacidade para dezasseis palavras de quatro bits.

Durante o teste foram introduzidos sequencialmente os valores compreendidos entre 0000 e 1111. Na forma de onda é possível observar que cada valor é armazenado numa posição diferente da memória, preenchendo sucessivamente as dezasseis posições disponíveis.

Inicialmente, o sinal **empty** encontra-se ativo, indicando que o *Ring Buffer* não contém dados armazenados. Após a primeira operação de escrita, este sinal é desativado. À medida que são introduzidos novos valores, as posições seguintes da memória são preenchidas até ser atingida a capacidade máxima da estrutura.

Após a escrita do décimo sexto elemento, o sinal **full** é ativado, indicando que não existem posições livres para o armazenamento de novos dados. Posteriormente, quando é realizada uma operação de leitura e uma posição volta a ficar disponível, o sinal **full** é desativado.

A simulação permitiu, assim, confirmar a correta escrita dos dados nas posições da memória RAM e o funcionamento dos sinais de estado **empty** e **full**, responsáveis por indicar, respetivamente, se o *Ring Buffer* se encontra vazio ou completamente preenchido.

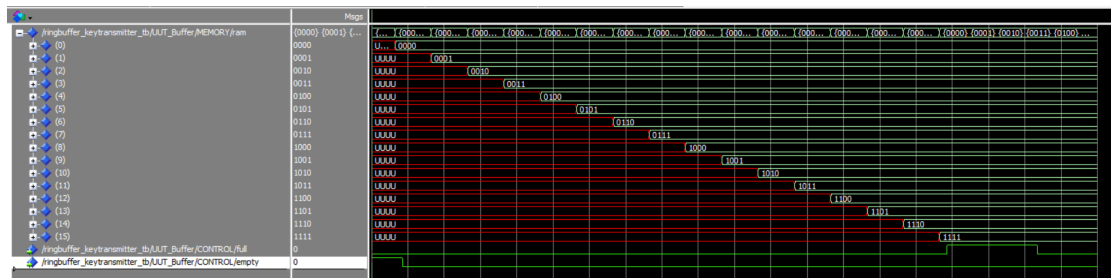


Figura 19: Simulação do preenchimento da memória RAM do *Ring Buffer* e ativação dos sinais **empty** e **full**.

3.2 Simulação do Software

Para validar o funcionamento da aplicação desenvolvida em Kotlin sem recorrer diretamente à placa FPGA, foi utilizado o simulador de hardware disponibilizado pelos docentes. Este ambiente permite emular os principais periféricos da Máquina de Venda de Bilhetes, nomeadamente o teclado matricial, o LCD, o *Coin Acceptor*, o *Keyboard Transmitter*, o *Ring Buffer* e o *Ticket Dispenser*.

A utilização do simulador permitiu testar a interação entre o módulo de controlo implementado em software e os diferentes componentes de hardware emulados. Foram verificadas as funcionalidades associadas ao modo de Venda e ao modo de Manutenção, incluindo a seleção de destinos, a aquisição e o cancelamento de bilhetes, a introdução de moedas e a consulta dos respetivos contadores.

Como exemplo dos testes realizados, apresenta-se na Figura 20 a ativação do modo de Manutenção. Este modo é selecionado através da chave de manutenção, representada pelo sinal M no simulador.

Após a ativação da chave, a aplicação abandona o funcionamento normal de venda e apresenta no LCD o menu correspondente às operações de manutenção. Na situação representada, é apresentada a opção de consulta dos contadores de moedas, permitindo ao utilizador aceder às informações relativas às moedas armazenadas no sistema.

Através do teclado, é possível selecionar as diferentes funcionalidades do modo de Manutenção, nomeadamente o teste da máquina, a consulta dos contadores de bilhetes, a consulta dos contadores de moedas, a inicialização dos contadores e o encerramento da aplicação.

A simulação permitiu confirmar a correta deteção da chave de manutenção, a alteração do modo de funcionamento da aplicação e a apresentação das respetivas opções no LCD.

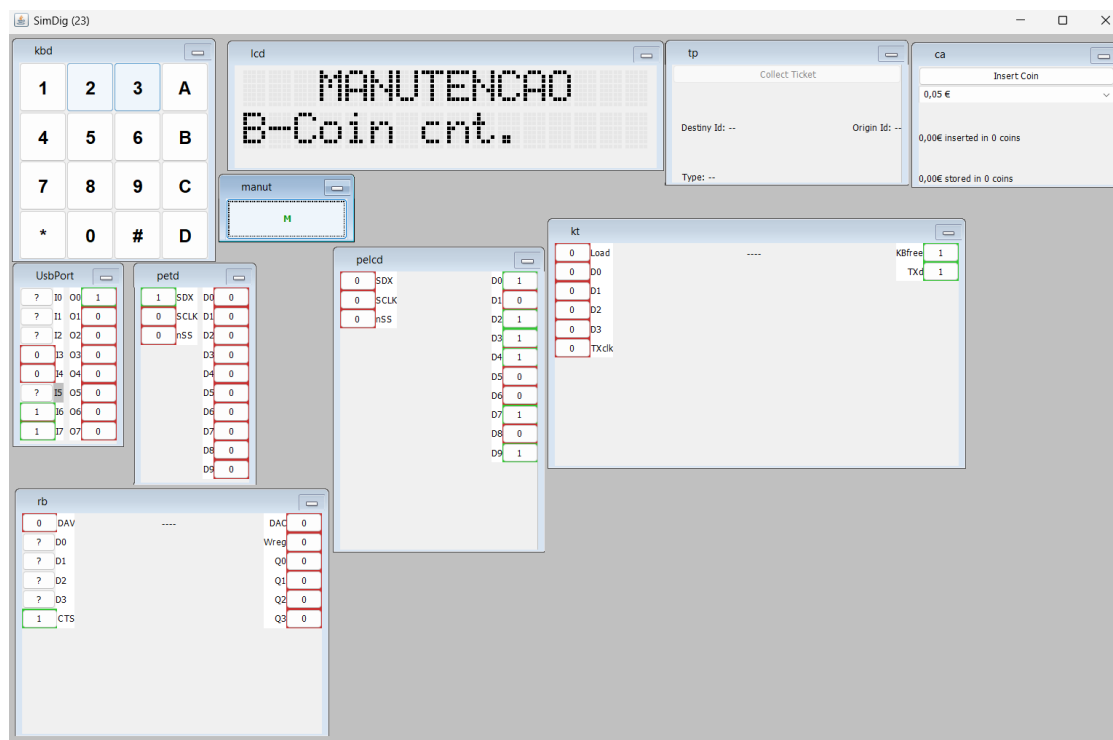


Figura 20: Ativação e apresentação do menu do modo de Manutenção no simulador de hardware.

4 Conclusões

O trabalho consistiu no desenvolvimento de uma máquina de venda de bilhetes chamada "Ticket Machine", conjugando componentes de hardware e software para garantir o funcionamento de todas as funções presentes na descrição do projeto. Para esse efeito, foi utilizada uma FPGA programada com a linguagem de descrição de hardware VHDL, a partir do ambiente Quartus Prime, e um computador com um software de controlo implementado em Kotlin. Os resultados experimentais confirmam o funcionamento do circuito de acordo com a descrição do funcionamento apresentada no enunciado.

Estes resultados foram obtidos a partir de testes realizados a cada componente, utilizando o ModelSim, uma extensão do programa Quartus que permite testar os componentes a partir dos ficheiros testbench que criámos, onde simulámos sinais e valores de entrada para observar os valores de saída e, assim, verificar o bom funcionamento de cada bloco que constitui o hardware do sistema.

Para realizar testes ao software em Kotlin, foi utilizado o ficheiro de simulação fornecido pelos docentes, que permite simular o software individualmente, para depois ser conectado ao hardware e testar o conjunto na placa de desenvolvimento FPGA DE10-Lite.

Os testes em hardware do sistema completo permitiram verificar o funcionamento total do sistema e também a realização de pequenos ajustes para o melhor funcionamento da máquina de venda de bilhetes implementada pelo grupo.

5 Trabalho Futuro

O trabalho pode ser melhorado em diversos aspetos, tanto ao nível do hardware como do software. No lado do hardware, uma possível otimização seria a remoção da estrutura de dados com disciplina FIFO (Ring Buffer) presente no módulo Keyboard Reader, uma vez que, no contexto atual do projeto, o sistema de controlo consome os dados praticamente ao mesmo ritmo a que são produzidos. Isto torna os buffers, na prática, dispensáveis, permitindo reduzir a complexidade do circuito e o consumo de recursos na FPGA.

Uma melhoria significativa do ponto de vista da experiência do utilizador seria a implementação de um menu deslizante ("scrollable") no ecrã LCD, permitindo o deslocamento horizontal e vertical entre as opções de estações através das teclas do teclado. Esta funcionalidade é especialmente útil quando o número de opções excede a capacidade de visualização simultânea do display, tornando a navegação mais intuitiva e eficiente. O sistema manteria internamente o controlo da posição atual do cursor (linha e coluna), atualizando dinamicamente o conteúdo do LCD conforme o utilizador se desloca. Adicionalmente, poderiam ser apresentados indicadores visuais, como setas, para sinalizar a existência de mais opções além das atualmente visíveis.

Outra melhoria fundamental seria a implementação de um sistema de devolução de trocos. Atualmente, a especificação da máquina de venda de bilhetes dita que o sistema não realiza trocos. Expandir a máquina de estados do módulo Coin Acceptor e a lógica do sistema de gestão em software (Kotlin) para calcular a diferença monetária e acionar a dispensa das respetivas moedas de troco aumentaria significativamente o realismo e a complexidade do projeto.

Por fim, ao nível do software, a adição de um sistema multilíngue no módulo TUI permitiria que

a interface suportasse vários idiomas além do português. Tratando-se de uma máquina de venda de bilhetes de comboio, acomodar utilizadores estrangeiros (com opções como Inglês e Espanhol) seria uma adição de grande valor prático à aplicação final.

Referências

- [1] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *KeyBoardReader datasheet*, 2026, ISEL.
- [2] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander LCD datasheet*, 2026, ISEL.
- [3] David Velez, Nuno Sebastião, Pedro Miguens Matutino, Rogério Rebelo, Rui Duarte *Port Expander Ticket Dispenser datasheet*, 2026, ISEL.
- [4] Terasic *DE10-lite User Manual*, 2020, Terasic.
- [5] Terasic *de10-Lite Exp. Board - Reference Manual*, 2022, Pedro Miguens Matutino

A Ticket Machine - Hardware

A.1 TicketMachine.vhd

```

library ieee;
use ieee.std_logic_1164.all;

entity TicketMachine is
    port(
        MClk      : in std_logic;
        Rst       : in std_logic;
        Tdelay    : in std_logic_vector(1 downto 0);
        linhas_in  : in std_logic_vector(3 downto 0);
        CollectTicket: in std_logic;
        Ca_Coin    : in std_logic;
        Ca_CoinID  : in std_logic_vector(2 downto 0);
        M         : in std_logic;
        D         : out std_logic_vector(9 downto 0);
        colunas_out : out std_logic_vector(3 downto 0);
        Ca_Collect  : out std_logic;
        Ca_Accept  : out std_logic;
        Ca_Eject   : out std_logic;
        K         : out std_logic_vector(3 downto 0);
        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5: out STD_LOGIC_VECTOR(7 downto 0)
    );
end TicketMachine;

architecture structural of TicketMachine is

    component KeyboardReader is
        port(
            MClk      : in std_logic;
            Rst       : in std_logic;
            Tdelay    : in std_logic_vector(1 downto 0);
            linhas_in  : in std_logic_vector(3 downto 0);
            Txclk     : in std_logic;
            K         : out std_logic_vector(3 downto 0);
            colunas_out : out std_logic_vector(3 downto 0);
            TxD       : out std_logic
        );
    end component;

    component UsbPort is
        port(
            inputPort : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
            outputPort : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
        );
    end component;

    component PELCD is
        port(
            SS : in std_logic;
            SCLK : in std_logic;
            SDX : in std_logic;
            Rst : in std_logic;
            D : out std_logic_vector(9 downto 0)
        );
    end component;

    component PETD is
        port(
            SS : in std_logic;
            SCLK : in std_logic;
            SDX : in std_logic;
            Rst : in std_logic;
            RT : out std_logic;
            Prt : out std_logic;
            O : out std_logic_vector(8 downto 5);
            D : out std_logic_vector(4 downto 1)
        );
    end component;

    component TICKET_DISPENSER is
        port (
            RT, Prt, CollectTicket: in STD_LOGIC;
            O, D: in STD_LOGIC_VECTOR(3 downto 0);
            Fn: out STD_LOGIC;
            HEX0, HEX1, HEX2, HEX3, HEX4, HEX5: out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;

end structural;

```

```

signal UsbPort_in    : std_logic_vector(7 downto 0);
signal UsbPort_out   : std_logic_vector(7 downto 0);
signal RT_out        : std_logic;
signal Prt_out       : std_logic;
signal O_out         : std_logic_vector(3 downto 0);
signal D_out         : std_logic_vector(3 downto 0);
signal K_out         : std_logic_vector(3 downto 0);

begin

UsbPort_in(0) <= Ca_CoinID(0) ;
UsbPort_in(1) <= Ca_CoinID(1) ;
UsbPort_in(2) <= Ca_CoinID(2) ;
UsbPort_in(3) <= Ca_Coin ;
--UsbPort_in(4) <= s_Kval ;
UsbPort_in(5) <= '0' ;
UsbPort_in(6) <= M ;
--UsbPort_in(7) <= '0' ;
--UsbPort_out(0) <= s_Ka ;
--UsbPort_out(1) <= '0' ;
--UsbPort_out(2) <= '0' ;
--UsbPort_out(3) <= '0' ;
--UsbPort_out(4) <= Ca_Accept ;
--UsbPort_out(5) <= Ca_Eject ;
--UsbPort_out(6) <= Ca_Collect ;
--UsbPort_out(7) <= '0' ;

KeyboardReader_V: KeyboardReader
port map(
    MClk      => MClk,
    K         => K_out,
    Rst       => Rst,
    Tdelay    => Tdelay,
    linhas_in => linhas_in,
    Txclk     => UsbPort_out(7),
    colunas_out => colunas_out,
    TxD       => UsbPort_in(7)
);

UsbPort_V: UsbPort
port map(
    inputPort  => UsbPort_in,
    outputPort => UsbPort_out
);

PETD_V: PETD
port map(
    SS      => UsbPort_out(3),
    SCLK    => UsbPort_out(1),
    SDX     => UsbPort_out(0),
    Rst     => Rst,
    RT      => RT_out,
    Prt     => Prt_out,
    O       => O_out,
    D       => D_out
);

PELCD_V: PELCD
port map(
    SS      => UsbPort_out(2),
    SCLK    => UsbPort_out(1),
    SDX     => UsbPort_out(0),
    Rst     => Rst,
    D       => D
);

TicketDispenser_V: TICKET_DISPENSER
port map(
    RT      => RT_out,
    Prt     => Prt_out,
    CollectTicket => CollectTicket,
    Fn      => UsbPort_in(4),
    O       => O_out,
    D       => D_out,
    HEX0    => HEX0,
    HEX1    => HEX1,
    HEX2    => HEX2,
    HEX3    => HEX3,
    HEX4    => HEX4,
    HEX5    => HEX5
);

Ca_Accept <= UsbPort_out(4);
Ca_Eject  <= UsbPort_out(5);
Ca_Collect <= UsbPort_out(6);
K <= K_out;

```

end structural;

A.2 Atribuição de Pinos

Entradas	pino
MClk	P11
Rst	F15
M	D12
CollectTicket	A14
Tdelay[0]	C10
Tdelay[1]	C11
Ca_Coin	C12
Ca_CoinId[0]	A12
Ca_CoinId[1]	B12
Ca_CoinId[2]	A13
linhas_in[0]	W5
linhas_in[1]	AA14
linhas_in[2]	W12
linhas_in[3]	AB12
Saídas	pino
colunas_out[0]	AB11
colunas_out[1]	AB10
colunas_out[2]	AA9
colunas_out[3]	AA8
Ca_Accept	D13
Ca_Eject	C13
Ca_Collect	E14
K[0]	A8
K[1]	A9
K[2]	A10
K[3]	B10
D[0]	W8
D[1]	AA15
D[2]	W13
D[3]	AB13
D[4]	Y11
D[5]	W11
D[6]	AA10
D[7]	Y8
D[8]	Y7
D[9]	V5
HEXs	Vários (ex: C14, C18, etc.)

Tabela 2: Atribuição de pinos para a função F [4]

B Ticket Machine - Software (*Control*)

B.1 Ticket Machine - App

Algoritmo 1: TicketMachineAPP

```
import java.time.LocalDateTime

object TicketMachineApp {

    private enum class State {
        WELCOME,
        CHOOSE_STATION,
        PAYMENT,
        PRINTING,
        WAIT_TICKET_COLLECTION,
        THANK_YOU
    }

    private var state = State.WELCOME
    private var roundTrip = false
    private var remainingPrice = 0
    private var paidAmount = 0

    private var maintenanceTicketIndex = 0
    private var maintenanceCoinIndex = 0

    private var maintenanceMenuIndex = 0
    private var lastMaintenanceMenuChange = 0L

    fun start() {
        HAL.init()
        CoinAcceptor.init()
        TD.init()
        TUI.init()

        Stations.init()
        CoinDeposit.init()

        state = State.WELCOME
        run()
    }

    private fun run() {
        while (true) {
            if (Maintenance.isActive()) {
                runMaintenanceMode()
            }

            when (state) {
                State.WELCOME -> runWelcome()
                State.CHOOSE_STATION -> runChooseStation()
                State.PAYMENT -> runPayment()
                State.PRINTING -> runPrinting()
                State.WAIT_TICKET_COLLECTION -> runWaitTicketCollection()
                State.THANK_YOU -> runThankYou()
            }

            Thread.sleep(10)
        }
    }

    private fun runWelcome() {
```



```

var lastShownMinute = -1

while (state == State.WELCOME) {
    if (Maintenance.isActive()) {
        return
    }

    val now = LocalDateTime.now()

    if (now.minute != lastShownMinute) {
        TUI.showWelcome(now)
        lastShownMinute = now.minute
    }

    val key = TUI.getKey()

    if (key != KBD.none) {
        state = State.CHOOSE_STATION
        drawStationChoice()
        return
    }

    Thread.sleep(20)
}

private fun runChooseStation() {
    val key = TUI.getKey()
    if (key == KBD.none) return

    when (key) {
        'A' -> {
            Stations.previous()
            drawStationChoice()
        }

        'B' -> {
            Stations.next()
            drawStationChoice()
        }

        in '0'..'9' -> {
            if (Stations.selectByDigit(key)) {
                drawStationChoice()
            }
        }

        '#' -> {
            Stations.clearDigitBuffer()
            roundTrip = false
            paidAmount = 0
            CoinDeposit.clearInsertedCoins()
            remainingPrice = Stations.currentOneWayPrice()
            state = State.PAYMENT
            drawPayment()
        }
    }
}

private fun runPayment() {
    val key = TUI.getKey()

    if (key != KBD.none) {
        when (key) {

```

```

        '*' -> {
            roundTrip = !roundTrip
            remainingPrice = Stations.currentTargetPrice(roundTrip) - paidAmount

            if (remainingPrice < 0) {
                remainingPrice = 0
            }

            drawPayment()
            return
        }

        '#' -> {
            if (paidAmount > 0) {
                CoinAcceptor.ejectCoins()
                paidAmount = 0
                CoinDeposit.clearInsertedCoins()
                remainingPrice = Stations.currentTargetPrice(roundTrip)
                TUI.showCanceledSale()
                Thread.sleep(2000)
            }

            state = State.WELCOME
            return
        }
    }

    }

    val coin = CoinAcceptor.getCoinValue()

    if (coin != null) {
        CoinAcceptor.acceptCoin()
        waitCoinRemoval()

        CoinDeposit.addInsertedCoin(coin)

        paidAmount += coin
        remainingPrice = Stations.currentTargetPrice(roundTrip) - paidAmount

        if (remainingPrice < 0) {
            remainingPrice = 0
        }

        drawPayment()

        if (remainingPrice == 0) {
            state = State.PRINTING
        }
    }
}

private fun runPrinting() {
    val station = Stations.current()

    TUI.showTicketFor(station.name)
    Thread.sleep(2000)

    TUI.showPrinting()

    TD.activatePrintingTicket(
        roundTrip = roundTrip,
        origin = Common.ORIGIN_ID,
        destination = Stations.printerDestinationId(station)
    )
}

```

```

CoinAcceptor.collectCoins()

CoinDeposit.collectInsertedCoins()
Stations.incrementSoldCurrent()

Stations.save()
CoinDeposit.save()

state = State.WAIT_TICKET_COLLECTION
}

private fun runWaitTicketCollection() {
    val station = Stations.current()

    TUI.showTicketReady(station.name)

    while (!TD.isFinished()) {
        Thread.sleep(20)
    }

    TD.deactivatePrintingTicket()

    state = State.THANK_YOU
}

private fun runThankYou() {
    TUI.showThankYou()
    Thread.sleep(5000)

    paidAmount = 0
    remainingPrice = 0
    roundTrip = false
    CoinDeposit.clearInsertedCoins()

    state = State.WELCOME
}

private fun runMaintenanceMode() {
    maintenanceMenuIndex = 0
    lastMaintenanceMenuChange = 0L

    updateMaintenanceMenu()

    while (Maintenance.isActive()) {
        updateMaintenanceMenu()

        val key = waitMaintenanceKey(100)

        if (!Maintenance.isActive()) {
            break
        }

        when (key) {
            KBD.none -> {
                // menu continua alternando
            }

            '#' -> {
                runMaintenanceTest()
                resetMaintenanceMenu()
            }

            'A' -> {

```

```

        maintenanceTicketIndex = 0
        showTicketCounters()
        resetMaintenanceMenu()
    }

    'B' -> {
        maintenanceCoinIndex = 0
        showCoinCounters()
        resetMaintenanceMenu()
    }

    'C' -> {
        confirmResetCounters()
        resetMaintenanceMenu()
    }

    'D' -> {
        confirmShutdown()
        resetMaintenanceMenu()
    }
}

Thread.sleep(20)
}

state = State.WELCOME
paidAmount = 0
remainingPrice = 0
roundTrip = false
CoinDeposit.clearInsertedCoins()

TUI.showWelcome(LocalDateTime.now())
}

private fun resetMaintenanceMenu() {
    maintenanceMenuIndex = 0
    lastMaintenanceMenuChange = 0L
    updateMaintenanceMenu()
}

private fun runMaintenanceTest() {
    Stations.resetSelection()
    roundTrip = false

    drawMaintenanceTestChoice()

    while (Maintenance.isActive()) {
        val key = waitMaintenanceKey(5000)

        if (!Maintenance.isActive()) {
            return
        }

        when (key) {
            KBD.none -> {
                TUI.showMaintenanceAborted()
                Thread.sleep(1500)
                return
            }

            'A' -> {
                Stations.previous()
                drawMaintenanceTestChoice()
            }
        }
    }
}

```

```

        'B' -> {
            Stations.next()
            drawMaintenanceTestChoice()
        }

        '*' -> {
            roundTrip = !roundTrip
            drawMaintenanceTestChoice()
        }

        '#' -> {
            val station = Stations.current()

            TUI.showPrinting()

            TD.activatePrintingTicket(
                roundTrip = roundTrip,
                origin = Common.ORIGIN_ID,
                destination = Stations.printerDestinationId(station)
            )

            TUI.showTicketReady(station.name)

            while (Maintenance.isActive() && !TD.isFinished()) {
                Thread.sleep(20)
            }

            TD.deactivatePrintingTicket()

            return
        }
    }
}

private fun updateMaintenanceMenu() {
    val now = System.currentTimeMillis()

    if (now - lastMaintenanceMenuChange >= Common.MAINTENANCE_MENU_DELAY) {
        TUI.showMaintenanceMenu(
            Common.MAINTENANCE_MENU_OPTIONS[maintenanceMenuIndex]
        )

        maintenanceMenuIndex =
            (maintenanceMenuIndex + 1) % Common.MAINTENANCE_MENU_OPTIONS.size

        lastMaintenanceMenuChange = now
    }
}

private fun drawMaintenanceTestChoice() {
    val station = Stations.current()

    TUI.showMaintenanceTest(
        stationNumber = station.number,
        stationName = station.name,
        price = Stations.currentTargetPrice(roundTrip),
        roundTrip = roundTrip
    )
}

private fun showTicketCounters() {
    val stations = Stations.all()

```

```

if (stations.isEmpty()) return

while (Maintenance.isActive()) {
    val station = stations[maintenanceTicketIndex]

    TUI.showTicketCounter(
        stationName = station.name,
        soldTickets = station.soldTickets
    )

    val key = waitMaintenanceKey(5000)

    if (!Maintenance.isActive()) {
        return
    }

    when (key) {
        KBD.none -> return

        'A' -> {
            maintenanceTicketIndex =
                if (maintenanceTicketIndex == 0) {
                    stations.lastIndex
                } else {
                    maintenanceTicketIndex - 1
                }
        }

        'B' -> {
            maintenanceTicketIndex =
                (maintenanceTicketIndex + 1) % stations.size
        }

        '#' -> return
    }
}

private fun showCoinCounters() {
    val coinValues = Common.COIN_VALUES

    while (Maintenance.isActive()) {
        val coin = coinValues[maintenanceCoinIndex]
        val count = CoinDeposit.getCoinCount(coin)

        TUI.showCoinCounter(
            coinValue = coin,
            numberCoins = count
        )

        val key = waitMaintenanceKey(5000)

        if (!Maintenance.isActive()) {
            return
        }

        when (key) {
            KBD.none -> return

            'A' -> {
                maintenanceCoinIndex =
                    if (maintenanceCoinIndex == 0) {
                        coinValues.lastIndex
                    }
            }
        }
    }
}

```

```

        } else {
            maintenanceCoinIndex = maintenanceCoinIndex - 1
        }
    }

    'B' -> {
        maintenanceCoinIndex =
            (maintenanceCoinIndex + 1) % coinValues.size
    }

    '#' -> return
}

}

private fun confirmResetCounters() {
    TUI.showResetConfirmation()

    val key = waitMaintenanceKey(5000)

    if (!Maintenance.isActive()) {
        return
    }

    if (key == '*') {
        Stations.resetSoldCounters()
        CoinDeposit.resetCounters()

        Stations.save()
        CoinDeposit.save()

        TUI.showCountersReset()
        Thread.sleep(2000)
    } else {
        TUI.showMaintenanceAborted()
        Thread.sleep(1500)
    }
}

private fun confirmShutdown() {
    TUI.showShutdownConfirmation()

    val key = waitMaintenanceKey(5000)

    if (!Maintenance.isActive()) {
        return
    }

    if (key == '*') {
        Stations.save()
        CoinDeposit.save()

        TUI.showShutdown()
        Thread.sleep(2000)

        kotlin.system.exitProcess(0)
    } else {
        TUI.showMaintenanceAborted()
        Thread.sleep(1500)
    }
}

private fun waitMaintenanceKey(timeout: Long): Char {
    val endTime = System.currentTimeMillis() + timeout

```

```

        while (Maintenance.isActive() && System.currentTimeMillis() < endTime) {
            val key = KBD.getKey()

            if (key != KBD.none) {
                return key
            }

            Thread.sleep(20)
        }

        return KBD.none
    }

    private fun drawStationChoice() {
        val s = Stations.current()

        TUI.showStationChoice(
            stationNumber = s.number,
            stationName = s.name,
            oneWayPrice = s.oneWayPrice
        )
    }

    private fun drawPayment() {
        TUI.showPaymentScreen(
            stationName = Stations.current().name,
            roundTrip = roundTrip,
            remaining = remainingPrice
        )
    }

    private fun waitCoinRemoval() {
        while (CoinAcceptor.hasCoin()) {
            Thread.sleep(20)
        }
    }
}

fun main() {
    TicketMachineApp.start()
}

```

B.2 M

Algoritmo 2: M

```

object Maintenance {
    /*
     * Botão/chave de manutenção M.
     *
     * No ficheiro .simul:
     *
     * m.out -> UsbPort.I6
     *
     * Portanto:
     * I6 = 1 shl 6 = 0x40
     */
    private const val M_MASK = 0x40

    fun isActive(): Boolean {
        return HAL.isBit(M_MASK)
    }
}

```


}

B.3 TUI

Algoritmo 3: TUI

```
import java.time.LocalDateTime

object TUI {

    fun init() {
        LCD.init()
        LCD.loadEuroSymbol(0)
        LCD.loadArrowUp(1)
        LCD.loadArrowDown(2)
        KBD.init()
    }

    fun getKey(): Char = KBD.getKey()

    fun waitAnyKey(): Char {
        while (true) {
            val k = KBD.getKey()
            if (k != KBD.none) return k
            Thread.sleep(50)
        }
    }

    fun write(text: String, lin: Int, col: Int, center: Boolean, clear: Boolean) {
        if (clear) LCD.clear()
        if (text.length <= LCD.COLS) {
            val startCol = if (center) center(text) else col
            LCD.cursor(lin, 0)
            LCD.write(text)
        } else {
            val line0Text = text.substring(0, LCD.COLS)
            val startCol0 = if (center) center(line0Text) else 0
            LCD.cursor(0, 0)
            LCD.write(line0Text)
            val remaining = text.substring(LCD.COLS)
            val line1Text = if (remaining.length > LCD.COLS) remaining.substring(0, LCD.COLS) else remaining
            val startCol1 = if (center) center(line1Text) else 0
            LCD.cursor(1, 0)
            LCD.write(line1Text)
        }
    }

    fun showWelcome(now: LocalDateTime) {
        LCD.clear()
        LCD.cursor(0, 0)
        LCD.write(center("Comprar Bilhete"))

        val text = "%02d/%02d/%04d %02d:%02d".format(
            now.dayOfMonth,
            now.monthValue,
            now.year,
            now.hour,
            now.minute
        )

        LCD.cursor(1, 0)
        LCD.write(cut16(text))
    }
}
```

```

fun showStationChoice(stationNumber: Int, stationName: String, oneWayPrice: Int) {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center(stationName))

    LCD.cursor(1, 0)
    LCD.write(stationNumber.toString().padStart(2, '0'))
    LCD.writeCharCode(1)
    LCD.writeCharCode(2)

    writeAmountRight(oneWayPrice)
}

fun showPaymentScreen(stationName: String, roundTrip: Boolean, remaining: Int) {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center(stationName))

    LCD.cursor(1, 0)
    LCD.writeCharCode(1)

    if (roundTrip) {
        LCD.writeCharCode(2)
    }

    writeAmountRight(remaining)
}

fun showPrinting() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("A␣imprimir..."))

    LCD.cursor(1, 0)
    LCD.write(center("Aguarde"))
}

fun showCanceledSale() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Venda␣cancelada"))

    LCD.cursor(1, 0)
    LCD.write(center("Reembolsando..."))
}

fun showTicketFor(stationName: String) {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Bilhete␣para"))

    LCD.cursor(1, 0)
    LCD.write(center(stationName))
}

fun showTicketReady(stationName: String) {
    LCD.clear()
}

```

```

        LCD.cursor(0, 0)
        LCD.write(center(stationName))

        LCD.cursor(1, 0)
        LCD.write(center("Retire bilhete"))
    }

    fun showThankYou() {
        LCD.clear()

        LCD.cursor(0, 0)
        LCD.write(center("Obrigado!"))

        LCD.cursor(1, 0)
        LCD.write(center("Boa viagem!"))
    }

    fun showMaintenanceMenu(optionText: String) {
        LCD.clear()

        LCD.cursor(0, 0)
        LCD.write(center("MANUTENCAO"))

        LCD.cursor(1, 0)
        LCD.write(cut16(optionText))
    }

    fun showMaintenanceTest(
        stationNumber: Int,
        stationName: String,
        price: Int,
        roundTrip: Boolean
    ) {
        LCD.clear()

        LCD.cursor(0, 0)
        LCD.write(cut16("TESTE $stationName"))

        LCD.cursor(1, 0)
        LCD.write(stationNumber.toString().padStart(2, '0'))

        /*
         * Tamb m usa setas, para ficar igual ao modo venda.
         */
        LCD.writeCharCode(1)

        if (roundTrip) {
            LCD.writeCharCode(2)
        }

        writeAmountRight(price)
    }

    fun showTicketCounter(
        stationName: String,
        soldTickets: Int
    ) {
        LCD.clear()

        LCD.cursor(0, 0)
        LCD.write(cut16(stationName))

        LCD.cursor(1, 0)
        LCD.write(cut16("Vendidos: $soldTickets"))
    }

```

```

}

fun showCoinCounter(
    coinValue: Int,
    numberCoins: Int
) {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write("Moeda: ")
    LCD.write(amountText(coinValue))
    LCD.writeCharCode(0)

    LCD.cursor(1, 0)
    LCD.write(cut16("Qtd: $numberCoins"))
}

fun showResetConfirmation() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Zerar cont.?"))

    LCD.cursor(1, 0)
    LCD.write(center("* confirm"))
}

fun showCountersReset() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Contadores"))

    LCD.cursor(1, 0)
    LCD.write(center("zerados"))
}

fun showShutdownConfirmation() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Desligar?"))

    LCD.cursor(1, 0)
    LCD.write(center("* confirm"))
}

fun showShutdown() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Desligando..."))
    LCD.clear()
}

fun showMaintenanceAborted() {
    LCD.clear()

    LCD.cursor(0, 0)
    LCD.write(center("Comando"))

    LCD.cursor(1, 0)
    LCD.write(center("abortado"))
}

```

```

}

private fun writeAmountRight(cents: Int) {
    val txt = amountText(cents)
    val start = (LCD.COLS - txt.length - 1).coerceAtLeast(0)

    LCD.cursor(1, start)
    LCD.write(txt)
    LCD.writeCharCode(0)
}

private fun amountText(cents: Int): String {
    val euros = cents / 100
    val rest = cents % 100

    return "$euros.${rest.toString().padStart(2, '0')}"
}

private fun center(text: String): String {
    val cut = cut16(text)
    val left = ((LCD.COLS - cut.length) / 2).coerceAtLeast(0)

    return " ".repeat(left) + cut
}

private fun cut16(text: String): String {
    return if (text.length <= 16) {
        text
    } else {
        text.substring(0, 16)
    }
}
}

```

B.4 Coin Acceptor

Algoritmo 4: Coin Acceptor

```

object CoinAcceptor {
    private const val COIN_MASK = 0x08    // I3
    private const val CID_MASK = 0x07    // I0..I2

    private const val ACCEPT_MASK = 0x10    // 04
    private const val EJECT_MASK = 0x20    // 05
    private const val COLLECT_MASK = 0x40    // 06

    fun init() {
        HAL.clrBits(ACCEPT_MASK or EJECT_MASK or COLLECT_MASK)
    }

    fun hasCoin(): Boolean {
        return HAL.isBit(COIN_MASK)
    }

    fun getCoinValue(): Int? {
        if (!hasCoin()) return null

        return when (HAL.readBits(CID_MASK)) {
            0 -> 5
            1 -> 10
            2 -> 20
            3 -> 50
            4 -> 100
        }
    }
}

```

```

        5 -> 200
        else -> null
    }
}

fun acceptCoin() {
    HAL.setBits(ACCEPT_MASK)
    Thread.sleep(200)
    HAL.clrBits(ACCEPT_MASK)
}

fun ejectCoins() {
    HAL.setBits(EJECT_MASK)
    Thread.sleep(2000)
    HAL.clrBits(EJECT_MASK)
}

fun collectCoins() {
    HAL.setBits(COLLECT_MASK)
    Thread.sleep(2000)
    HAL.clrBits(COLLECT_MASK)
}
}

//fun main() {
//    CoinAcceptor.init()
//    while (true) {
//        val value = CoinAcceptor.getCoinValue()
//        if (value != null) {
//            println("Moeda detectada : $value cent")
//            CoinAcceptor.acceptCoin()
//        }
//        Thread.sleep(100)
//    }
//}

```

B.5 Coin Deposit

Algoritmo 5: Coin Deposit

```

object CoinDeposit {
    private var coinCounters = mutableMapOf<Int, Int>()
    private val insertedCoins = mutableListOf<Int>()

    fun init() {
        coinCounters = FileAccess.loadCoins()
    }

    fun addInsertedCoin(coin: Int) {
        insertedCoins.add(coin)
    }

    fun collectInsertedCoins() {
        for (coin in insertedCoins) {
            coinCounters[coin] = (coinCounters[coin] ?: 0) + 1
        }

        insertedCoins.clear()
    }

    fun clearInsertedCoins() {
        insertedCoins.clear()
    }
}

```

```

    }

    fun getCoinCount(coin: Int): Int {
        return coinCounters[coin] ?: 0
    }

    fun resetCounters() {
        for (coin in Common.COIN_VALUES) {
            coinCounters[coin] = 0
        }
    }

    fun save() {
        FileAccess.saveCoins(coinCounters)
    }
}

```

B.6 Statistics

B.7 File Access

Algoritmo 6: File Access

```

import java.io.BufferedReader
import java.io.File
import java.io.FileReader
import java.io.PrintWriter

object FileAccess {

    fun loadStations(filename: String = "stations.txt"): MutableList<Station> {
        val stations = mutableListOf<Station>()

        val file = File(filename)
        if (!file.exists()) return stations

        BufferedReader(FileReader(file)).use { br ->
            var number = 1

            while (true) {
                val line = br.readLine() ?: break
                val parts = line.split(";")

                if (parts.size >= 3) {
                    val price = parts[0].trim().toInt()
                    val sold = parts[1].trim().toInt()
                    val name = parts[2].trim()

                    stations.add(
                        Station(
                            number = number,
                            name = name,
                            oneWayPrice = price,
                            soldTickets = sold
                        )
                    )

                    number++
                }
            }
        }

        return stations
    }
}

```

```

    }

    fun saveStations(
        stations: List<Station>,
        filename: String = "stations.txt"
    ) {
        PrintWriter(filename).use { out ->
            for (station in stations) {
                out.println("${station.oneWayPrice};${station.soldTickets};${station.name}")
            }
        }
    }

    fun loadCoins(filename: String = "CoinDeposit.txt"): MutableMap<Int, Int> {
        val coins = mutableMapOf(
            5 to 0,
            10 to 0,
            20 to 0,
            50 to 0,
            100 to 0,
            200 to 0
        )

        val file = File(filename)
        if (!file.exists()) return coins

        BufferedReader(FileReader(file)).use { br ->
            while (true) {
                val line = br.readLine() ?: break
                val parts = line.split(";")

                if (parts.size >= 2) {
                    val coinValue = parts[0].trim().toInt()
                    val numberCoins = parts[1].trim().toInt()

                    coins[coinValue] = numberCoins
                }
            }
        }

        return coins
    }

    fun saveCoins(
        coins: Map<Int, Int>,
        filename: String = "CoinsDeposit.txt"
    ) {
        PrintWriter(filename).use { out ->
            for (coin in Common.COIN_VALUES) {
                out.println("$coin;${coins[coin]?:0}")
            }
        }
    }
}

```